This switch to Namespaces had a problem: if someone had an old Rake build file lying around—one that still referenced the earlier, non-Namespaced class names—that file wouldn't work with an upgraded version of Rake. For this reason, Rake maintained compatibility with older build files for a while. It did so by providing a command-line option named classic-namespace that loaded an additional source file. This source file assigned the new, safer constant names to the old, unsafe ones:

gems/rake-0.9.2.2/lib/rake/classic_namespace.rb

```
Task = Rake::Task
FileTask = Rake::FileTask
FileCreationTask = Rake::FileCreationTask
# ...
```

When this file was loaded, both Task and Rake::Task ended up referencing the same instance of Class, so a build file could use either constant to refer to the class. A few versions afterwards, Rake assumed that all users had migrated their build file, and it removed the option.

Enough digression on constants. Let's go back to objects and classes, and wrap up what you've just learned.

Objects and Classes Wrap-Up

What's an object? It's a bunch of instance variables, plus a link to a class. The object's methods don't live in the object—they live in the object's class, where they're called the *instance methods* of the class.

What's a class? It's an object (an instance of Class), plus a list of instance methods and a link to a superclass. Class is a subclass of Module, so a class is also a module. If this is confusing, look back at Figure 2, *Classes are just objects*, on page 21.

These are instance methods of the Class class. Like any object, a class has its own methods, such as new. Also like any object, classes must be accessed through references. You already have a constant reference to each class: the class's name.

"That's pretty much all there is to know about objects and classes," Bill says. "If you can understand this, you're well on your way to understanding metaprogramming. Now, let's turn back to the code."